

Ejercicio 1.

```
[5]: miLength :: [a] -> Int
miLength [] = 0
miLength (_:xs) = 1 + miLength xs
miLength [10,9,8,7,6,5,4,3,2,1]

10
```

```
[6]: miLast :: [a] -> a
miLast [x] = x
miLast (_:xs) = miLast xs
miLast [10,9,8,7,6,5]

5
```

```
[24]: duplicar :: [Int] -> [Int]
duplicar xs = map (* 2) xs
duplicar [4,3,2,1]
```

Eta reduce

Found:

Why Not:

```
duplicar xs = map (* 2) xs
duplicar = map (* 2)

[8,6,4,2]
```

```
eliminarPares :: [Int] -> [Int]
eliminarPares xs = filter odd xs
eliminarPares [12,13,14,15,16,17,18,19,20]
```

Eta reduce

Found:

Why Not:

```
eliminarPares xs = filter odd xs
eliminarPares = filter odd

[13,15,17,19]
```

Ejercicio 2.

1. Compresión de listas.

```
cuadrados = [ x * x | x <- [1..10] ]  
divisiblesPor3 = [ x | x <- [1..30], x `mod` 3 == 0 ]  
paresOrdenados = [ (x, y) | x <- [1..5], y <- [1..5], x < y ]
```

```
cuadrados  
divisiblesPor3  
paresOrdenados
```

```
[1,4,9,16,25,36,49,64,81,100]  
[3,6,9,12,15,18,21,24,27,30]  
[(1,2),(1,3),(1,4),(1,5),(2,3),(2,4),(2,5),(3,4),(3,5),(4,5)]
```

2. Función que devuelve los divisores de un número.

```
divisores :: Int -> [Int]  
divisores n = [ d | d <- [1..n], n `mod` d == 0 ]
```

```
divisores 12  
divisores 7  
divisores 1
```

```
[1,2,3,4,6,12]  
[1,7]  
[1]
```

3. Número perfecto.

```
divisoresPropios :: Int -> [Int]
divisoresPropios n = [ d | d <- [1..n-1], n `mod` d == 0 ]

esPerfecto :: Int -> Bool
esPerfecto n = sum (divisoresPropios n) == n

divisoresPropios 6
divisoresPropios 12
esPerfecto 6
esPerfecto 28
esPerfecto 12

[1,2,3]
[1,2,3,4,6]
True
True
False
```

Ejercicio 3.

1. Reescribir sin recursión explícita.

```
sumaLista = foldr (+) 0
mayoresQue10 xs = length (filter (> 10) xs)
productoLista = foldr (*) 1
celsiusAFahrenheit = map (\c -> c * 9 / 5 + 32)
ajustarImpares = map (\x -> if odd x then x + 5 else x)
```

```
sumaLista [1,2,3]
mayoresQue10 [5,11,20,3]
productoLista [2,3,4]
celsiusAFahrenheit [0,100]
ajustarImpares [1,2,3,4]
```

Use sum

Found: **Why Not:**

foldr (+) 0sum

Use product

Found: **Why Not:**

foldr (*) 1product

```
6
2
24
[32.0,212.0]
[6,2,8,4]
```

2. Definir usando Filter.

```
nombresLargos = filter (\n -> length n > 5)
soloNegativos = filter (< 0)
edadesUniversitarios = filter (\e -> e >= 18 && e <= 25)
```

```
nombresLargos ["Ana","Alejandro","Maria","Fernanda"]
soloNegativos [3,-1,0,-7]
edadesUniversitarios [17,18,22,25,26]
```

```
["Alejandro","Fernanda"]
[-1,-7]
[18,22,25]
```

3. Definir usando filter y map.

```

agregarIVA = map (\p -> if p >= 1000 then p * 1.19 else p)
longitudesVocales = map length . filter (\w -> head w `elem` "aeiouAEIOU")
cuadradosPares = map (\x -> x * x) . filter even

```

```

agregarIVA [500, 1000, 2000]
longitudesVocales ["arbol", "casa", "Elena", "sol"]
cuadradosPares [1,2,3,4]

```

```

[500.0,1190.0,2380.0]
[5,5]
[4,16]

```

4. Definir usando any o all.

```

hayNegativo = any (< 0)
todosPares = all even
import Data.Char (isUpper)

nombresCorrectos = all (\n -> isUpper (head n))

```

```

hayNegativo [3, -1, 5]
hayNegativo [3, 1, 5]
todosPares [2, 4, 6]
todosPares [2, 3, 6]
nombresCorrectos ["Ana", "Luis"]
nombresCorrectos ["Ana", "luis"]

```

```

True
False
True
False
True
False

```

5. Definir usando takeWhile o dropWhile.

```

menoresQue100 = takeWhile (< 100)
sinCerosIniciales = dropWhile (== 0)

```

```

menoresQue100 [5, 50, 99, 100, 200]
sinCerosIniciales [0, 0, 3, 0, 5]

```

```

[5,50,99]
[3,0,5]

```

Ejercicio 4.

```
procesarLecturas :: (Double -> Double) -> [Double] -> [Double]
procesarLecturas f xs = map f xs

celsiusAFahrenheit :: Double -> Double
celsiusAFahrenheit c = c * 9 / 5 + 32

crearFiltro :: (a -> Bool) -> ([a] -> [a])
crearFiltro p = filter p

soloPares :: [Int] -> [Int]
soloPares = crearFiltro even

enRango :: Int -> Int -> (Int -> Bool)
enRango minimo maximo = \x -> x >= minimo && x <= maximo

entre5y10 :: Int -> Bool
entre5y10 = enRango 5 10

potenciador :: Int -> (Int -> Int)
potenciador n = \x -> x ^ n

potencia5 :: Int -> Int
potencia5 = potenciador 5
```

```
: procesarLecturas celsiusAFaranheit [0,10,25]
```

```
[32.0,50.0,77.0]
```

```
: soloPares[1..20]
```

```
[2,4,6,8,10,12,14,16,18,20]
```

```
: entre5y10 7
```

```
True
```

```
: potencia5 7
```

```
16807
```

Ejercicio 5.

1. Funciones de transición de estados

```
transferirAB :: Estado -> Estado
transferirAB (a, b) =
    let cant = min a (capB - b)
    in (a - cant, b + cant)

transferirBA :: Estado -> Estado
transferirBA (a, b) =
    let cant = min b (capA - a)
    in (a + cant, b - cant)
```

2. Heurística admisible

```
heuristica :: Estado -> Costo
heuristica (4, 0) = 0
heuristica _      = 1
```

3. Definición de Inicio y Meta

```
inicio :: Estado
inicio = (0, 0)

meta :: Estado
meta = (4, 0)
```

4. Código Completo Integrado

```
import Data.List (sortOn)

type Estado = (Int, Int)
type Nodo = Estado
type Costo = Int
type Grafo = [(Nodo, [(Nodo, Costo)])]

capA, capB :: Int
capA = 5
capB = 3

inicio :: Estado
inicio = (0, 0)

meta :: Estado
meta = (4, 0)

transferirAB :: Estado -> Estado
transferirAB (a, b) =
    let cant = min a (capB - b)
    in (a - cant, b + cant)

transferirBA :: Estado -> Estado
transferirBA (a, b) =
    let cant = min b (capA - a)
    in (a + cant, b - cant)

vecinos :: Estado -> [(Estado, Costo)]
vecinos (a, b) =
    [ ((capA, b), 1)
    , ((a, capB), 1)
    , ((0, b), 1)
    , ((a, 0), 1)
    , (transferirAB (a, b), 1)
    , (transferirBA (a, b), 1)
    ]

heuristica :: Estado -> Costo
heuristica (4, 0) = 0
heuristica _      = 1

aStar :: (Nodo -> [(Nodo, Costo)]) -> (Nodo -> Costo) -> Nodo -> Nodo -> [(Nodo, Costo)]
aStar sucesores heur inicioObj metaObj = buscar [(inicioObj, 0)] []
    where
        buscar [] _ = []
        buscar ((nodo, costo):cola) visitados
            | nodo == metaObj = [(nodo, costo)]
            | nodo `elem` visitados = buscar cola visitados
            | otherwise =
                let nuevos = [ (v, costo + c) | (v, c) <- sucesores nodo, v `notElem` visitados ]
                ordenados = sortOn (\(v, c) -> c + heur v) (cola ++ nuevos)
                in (nodo, costo) : buscar ordenados (nodo : visitados)

main :: IO ()
main = print (aStar vecinos heuristica inicio meta)
```

Retrospectiva

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes?
(Horas/Hombre)

Matias: 2,5 horas

Samuel: 1 hora

2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?

Terminado, todos los puntos completados

3. ¿Cuál consideran fue el mayor logro? ¿Por qué?

Entender mejor las estructuras.

4. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

Coordinar el trabajo en grupo.

5. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Comunicación y ayuda a resolver ejercicios.

6. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

Documentación de Haskell.